Programación declarativa @ URJC

Programación funcional

Curso 23-24, convocatoria ordinaria (30 de octubre de 2023)

Campus de Vicálvaro

Preámbulo

In [2]:
```
import $ivy.`org.scalatest::scalatest:3.0.8`
import org.scalatest._
```

Out[2]:
```
import $ivy.$

import org.scalatest._
```

Algunas definiciones de tipos y funciones auxiliares

In [3]:
```
sealed abstract class Tree[A]
case class Empty[A]() extends Tree[A]
case class Node[A](left: Tree[A], root: A, right: Tree[A]) extends Tree[A]

object Tree{
    def void[A]: Tree[A] = Empty()
    def leaf[A](a: A): Node[A] = Node(Empty(), a, Empty())
    def right[A](a: A, tree: Tree[A]): Node[A] = Node(Empty(), a, tree)
    def left[A](tree: Tree[A], a: A): Node[A] = Node(tree, a, Empty())
    def node[A](left: Tree[A], a: A, right: Tree[A]): Node[A] = Node(left, a
}

import Tree._

def foldTree[A, B](tree: Tree[A])(empty: B)(node: (B, A, B) => B): B =
    tree match {
        case Empty() => empty
        case Node(left, root, right) => node(foldTree(left)(empty)(node), ro
    }

def map[A, B](tree: Tree[A])(f: A => B): Tree[B] =
    foldTree(tree)(Empty(): Tree[B]){
        (leftMapped, root, rightMapped) =>
            Node(leftMapped, f(root), rightMapped)
    }
```

Out[3]:
```
defined class Tree
defined class Empty
defined class Node
defined object Tree
import Tree._


defined function foldTree
defined function map
```

In [52]:
```
object Signatures{
    abstract class List[A]{

        // Common HOFs
        def foldRight[B](nil: B)(cons: (A, B) => B): B
        def foldLeft[B](initial: B)(update: (B, A) => B): B
        def map[B](f: A => B): List[B]
        def flatMap[B](f: A => List[B]): List[B]
        def filter(f: A => Boolean): List[A]
        def forall(pred: A => Boolean): Boolean
        def exists(pred: A => Boolean): Boolean

        // Returns the number of elements of this list
        def length: Int

        def reverse: List[A]
    }
}
```

Out[52]: defined object Signatures

Definiciones auxiliares sobre la correspondencia Curry-Howard y Tipos Algebraicos de Datos

In [5]:
```
trait Isomorphic[A, B]{

    def from(a: A): B

    def to(b: B): A

    def law1(a: A): Boolean =
        to(from(a)) == a

    def law2(b: B): Boolean =
        from(to(b)) == b
}
```

Out[5]: defined trait Isomorphic

Algebraic data type	Constructor	Destructor

(exponents)
`A => B`
	`{ a: A => ??? : B }: (A => B)`	`val f: A => B = ???`
		`val a: A = ???`
		`f(a): B`

(products)
`(A, B)`
	`(??? : A, ??? : B): (A, B)`	`val t: (A, B) = ???`
		`t._1 : A`
		`t._2 : B`

(sums)
`Either[A, B]`
	`Left(??? : A): Either[A, B]`
	`Right(??? : B): Either[A, B]`
		`val e: Either[A, B] = ???`
		`e match {`
		`  case Left(a: A) => ??? : C`
		`  case Right(b: B) => ??? : C`
		`}`

(0) `Nothing`		`val n: Nothing = ???` `n: C`

In [6]:
```
type Not[P] = P => Nothing
```

Out[6]:
```
defined type Not
```

Ejercicio 1

(2 puntos)

a) (1 punto) Utiliza la correspondencia de Curry-Howard para demostrar la siguiente tautología de la lógica proposicional intuicionista:

$$\vdash \neg\neg p \vee \neg\neg q \rightarrow \neg\neg(p \vee q)$$

In [58]:
```
// Escribe tu solución en esta celda
            //Either[[[P=> Nothing] => Nothing],[[Q=> Nothing] => Nothing]]
def proof[P, Q]: Either[Not[Not[P]], Not[Not[Q]]] => Not[Not[Either[P, Q]]]
    f => e => f match{
        case Left(np) => (e => np(Left(p)))
        case Right(nq) => (e => nq(Left(q)))
    }
```

Out[58]:
```
defined function proof
```

b) (1 punto) Se desea utilizar la correspondencia de Curry-Howard para demostrar la validez del siguiente argumento de la lógica clásica proposicional:

$$\{p \wedge q \rightarrow r \vee s, \neg(t \wedge \neg p),\ t,\ t \rightarrow q, q \rightarrow t\} \vdash_c \neg r \rightarrow s$$

Para ello, utiliza como premisa adicional la ley del tercio excluso aplicada a la proposición p, es decir, demuestra la validez intuicionista del siguiente argumento lógico:

$$\{p \vee \neg p, p \wedge q \rightarrow r \vee s, \neg(t \wedge \neg p),\ t,\ t \rightarrow q, q \rightarrow t\} \vdash \neg r \rightarrow s$$

In [ ]:
```
// Escribe tu solución en esta celda
def proof [P, Q, R, S, T] (p1: Either[P, Not[P]],
                           p2: ((P, Q)) => Either[R, S],
                           p3: Not[(T, Not[P])],
                           p4: T,
                           p5: T => Q,
                           p6: Q => T): Not[R] => S=
    r => p1 match{
        case Left(p) => Right(p2(p, p5(p4)))
        case Right(np) => p3((p4, np))
    }
```

Ejercicio 2

(1,5 puntos)

Demuestra que, para todo tipo `A`, los tipos de datos `(Unit => Boolean) => Unit => A` y `(A, A)` son isomorfos:

In [56]:
```
// Escribe tu solución en esta celda, utilizando el identificador `Iso`
// como nombre de la clase que implementa el isomorfismo.

class Iso[A] extends Isomorphic[(Unit => Boolean) => Unit => A , (A, A)]{
    def from(a: (Unit => Boolean) => (Unit => A)): (A, A) =
       ???
     a match{
        case (None => true) => false
        case (None => false) => false
        case (Some(a) => true) => true
        case (Some(a) => false) => true
    }
    def to(b: (A, A)): (Unit => Boolean) => Unit => A =
        b match{
            case (None, Some(a)) => None
            case (None, None) => None
            case (Some(a), None) => Some(a)
            case (Some(a), Some(a)) => Some(a)
        }
```

Out[56]:
```
defined class Iso
```

La implementación deberá satisfacer las leyes de los isomorfismos, y en particular el siguiente test:

In [20]:
```
class TestIso(iso: Isomorphic[(Unit => Boolean) => Unit => Int, (Int, Int)])

    "law2" should "hold" in {
        iso.law2((1,3)) shouldBe true
        iso.law2((2,4)) shouldBe true
    }

    "law1" should "hold" in {
        iso.law1(f => _ => if (f(())) 3 else 4) shouldBe true
        iso.law1(f => _ => if (f(())) 6 else 7) shouldBe true
    }
}
```

Out[20]:
```
defined class TestIso
```

In [ ]:
```
run(new TestIso(new Iso[Int]{
    override def law1(b: (Unit => Boolean) => Unit => Int): Boolean =
        (to(from(b))(_ => false)(()) == b(_ => false)(())) &&
        (to(from(b))(_ => true)(()) == b(_ => true)(()))
}))
```

Ejercicio 3

(3 puntos)

La función `isSorted[A]` comprueba si una lista de elementos de tipo `A` se encuentra ordenada de acuerdo a una relación de orden `(A, A) => Boolean` dada. Por ejemplo, la función satisface los siguientes casos de prueba:

In [41]:
```
class TestSorted(isSorted: List[Int] => ((Int, Int) => Boolean) => Boolean)

    "sorted" should "work" in {

        isSorted(Nil)(_ < _) shouldBe true
        isSorted(List(1))(_ < _) shouldBe true
        isSorted(List(0,1))(_ < _) shouldBe true
        isSorted(List(0,1,2,3,4))(_ < _) shouldBe true
        isSorted(List(1,0))(_ < _) shouldBe false
        isSorted(List(1,2,0))(_ < _) shouldBe false
        isSorted(List(3,1,2))(_ < _) shouldBe false
    }
}
```

Out[41]:
```
defined class TestSorted
```

a) (1,5 puntos) Implementa la función `isSorted` mediante recursión final.

In [51]:
```
// Escribe tu solución en esta celda
def isSorted(l: List[Int], /*=>*/ n: ((Int, Int)) => Boolean): Boolean = {
  @annotation.tailrec
  def aux(l: List[Int])(devolver: Boolean): Boolean =
    l match {
        case List() => devolver // Lista vacía --> isSorted(Nil)(_ < _) shou
        case _ :: List() => devolver
        case (h1 :: h2 :: t)
            if n(h1, h2) =>
                aux(h2 :: t)(devolver)
        case _ => false
  }
  aux(l)(true)
}
```

Out[51]:
```
defined function isSorted
```

In [ ]:
```
run(new TestSorted(isSorted))
```

b) (1,5 puntos) Implementa la función `isSorted` mediante la función `foldLeft` .

In [ ]:
```
// Escribe tu solución en esta celda
///def foldLeft[B](initial: B)(update: (B, A) => B): B
def isSortedFL(l: List[Int], n: ((Int, Int)) => Boolean) : Boolean =
    l.foldLeft(true)({ //acc, elem
        l match{
           case(h1 :: h2 :: t) => isSortedFL(t, (n(h1, h2)))
        }
    })
```

In [ ]:
```
run(new TestSorted(isSorted))
```

Ejercicio 4

(3,5 puntos)

La función `heads` recibe un árbol cuyos nodos almacenan listas de elementos, y devuelve un árbol con la misma estructura donde los nodos almacenan el primer elemento de cada lista, *siempre y cuando todas las listas de elementos almacenadas en los nodos del árbol*

In [26]:
```
class TestHeads(heads: Tree[List[Int]] => Option[Tree[Int]]) extends FlatSpe

    "heads" should "work" in {
        heads(node(leaf(List(1,2,3)), List(5), leaf(List(10,0)))) shouldBe
            Some(node(leaf(1), 5, leaf(10)))

        heads(node(leaf(Nil), List(5,3), leaf(List(10)))) shouldBe
            None

        heads(node(leaf(List(1,2)), List(), leaf(List(10)))) shouldBe
            None

        heads(node(leaf(List(1,2)), List(5,3), leaf(List()))) shouldBe
            None

        heads(node(node(leaf(List(1,2)), List(6), leaf(List(3,4,2))), List(5
            Some(node(node(leaf(1), 6, leaf(3)), 5, leaf(2)))
    }
}
```

Out[26]:
```
defined class TestHeads
```

La función `divide` recibe un número entero y un árbol de números enteros, y devuelve un árbol con la misma estructura cuyos nodos resultan de dividir el número recibido como entrada y el número almacenado en el nodo del árbol, *siempre y cuando no exista ningún cero almacenado en algún nodo del árbol*. Por ejemplo:

In [27]:
```
class TestDivide(divide: Int => Tree[Int] => Option[Tree[Int]]) extends Flat

    "divide" should "work" in {
        divide(10)(node(leaf(1), 5, leaf(10))) shouldBe
            Some(node(leaf(10), 2, leaf(1)))

        divide(10)(node(leaf(0), 5, leaf(10))) shouldBe
            None

        divide(10)(node(leaf(1), 5, leaf(0))) shouldBe
            None

        divide(10)(node(leaf(1), 0, leaf(10))) shouldBe
            None

        divide(30)(node(node(leaf(1), 6, leaf(3)), 5, leaf(2))) shouldBe
            Some(node(node(leaf(30), 5, leaf(10)), 6, leaf(15)))
    }

}
```

Out[27]:
```
defined class TestDivide
```

Considérense las siguientes implementaciones de las funciones `heads` y `divide` basadas en un mismo patrón de diseño:

In [28]:
```
def heads[A](t: Tree[List[A]]): Option[Tree[A]] =
    t match {
        case Empty() =>
            Some(Empty())
        case Node(left, b, right) =>
            (heads(left), b match { case Nil => None; case h :: _ => Some(h)
                case (Some(left), Some(r), Some(right)) =>
                    Some(Node(left, r, right))
                case _ =>
                    None
            }
    }
```

Out[28]:
```
defined function heads
```

In [29]:
```
run(new TestHeads(heads))
```

```
cmd25$Helper$TestHeads:
heads
- should work
```

In [30]:
```
def divide(a: Int)(t: Tree[Int]): Option[Tree[Int]] =
    t match {
        case Empty() =>
            Some(Empty())
        case Node(left, b, right) =>
            (divide(a)(left), if (b == 0) None else Some(a/b), divide(a)(rig
                case (Some(left), Some(r), Some(right)) =>
                    Some(Node(left, r, right))
                case _ =>
                    None
            }
    }
```

Out[30]:
```
defined function divide
```

In [31]:
```
run(new TestDivide(divide))
```

```
cmd26$Helper$TestDivide:
divide
- should work
```

a) (1,5 puntos) Implementa una función de orden superior, denominada `traverse`, que encapsule el patrón de diseño utilizado en las implementaciones de las funciones `heads` y `divide`.

In [ ]:
```
// Escribe tu solución en esta celda
def traverse[A]/*(a: Int)*/(t: Tree[List[A]]): Option[Tree[A]] =
    t match{
        case Empty() => Some(Empty())
        case Node(left, b, right) =>
            (traverse/*(a)*/(Left),
                b match{
                    case Nil => None //if (b==0) => None
                    case h:: _ =>  Some(h) //else Some(a/b)
                }, reverse/*(a)*/(Right))
            match{
                case (Some(left), Some(r), Some(right)) =>
                    Some(Node(left, r, right))
                case _ => None
            }
    }
```

b) (1,5 puntos) Implementa de una manera más modular las funciones `heads` y `divide` mediante la función `traverse` .

In [ ]:
```
def heads[A](t: Tree[List[A]]): Option[Tree[A]] =
    traverse(_)(t)
```

In [ ]:
```
run(new TestHeads(heads))
```

In [ ]:
```
def divide(a: Int)(t: Tree[Int]): Option[Tree[Int]] =
    traverse(a)(t)
```

In [ ]:
```
run(new TestDivide(divide))
```

c) (0,5 puntos) Implementa la función `traverse` mediante la función `map` sobre árboles binarios, cuya definición se puede encontrar en el preámbulo del examen, y la función `sequence` definida a continuación:

In [32]:
```
def sequence[A](t: Tree[Option[A]]): Option[Tree[A]] =
    t match {
        case Empty() =>
            Some(Empty())
        case Node(left, a, right) =>
            (sequence(left), a, sequence(right)) match {
                case (Some(left), Some(a), Some(right)) =>
                    Some(Node(left, a, right))
                case _ =>
                    None
            }
    }
```

Out[32]:
```
defined function sequence
```

In [ ]:
```
// Escribe tu solución en esta celda
def map[A, B](tree: Tree[A])(f: A => B): Tree[B] =
    foldTree(tree)(Empty(): Tree[B]){
        (leftMapped, root, rightMapped) =>
            Node(leftMapped, f(root), rightMapped)
    }

// Escribe tu solución en esta celda
def traverse[A]/*(a: Int)*/(t: Tree[List[A]]): Option[Tree[A]] =
    map(t)(Empty(): Option[Tree[A]]){
        case Node(left, b, right) => sequence(t)
            (sequence/*(a)*/(Left),
                b match{
                    case Nil => None //if (b==0) => None
                    case h:: _ =>  Some(h) //else Some(a/b)
                }, sequence/*(a)*/(Right))
            match{
                case (Some(left), Some(r), Some(right)) => sequence(Node(le
                case _ => None
            }
    }
```